

Application Layer: Overview (7 of 7)

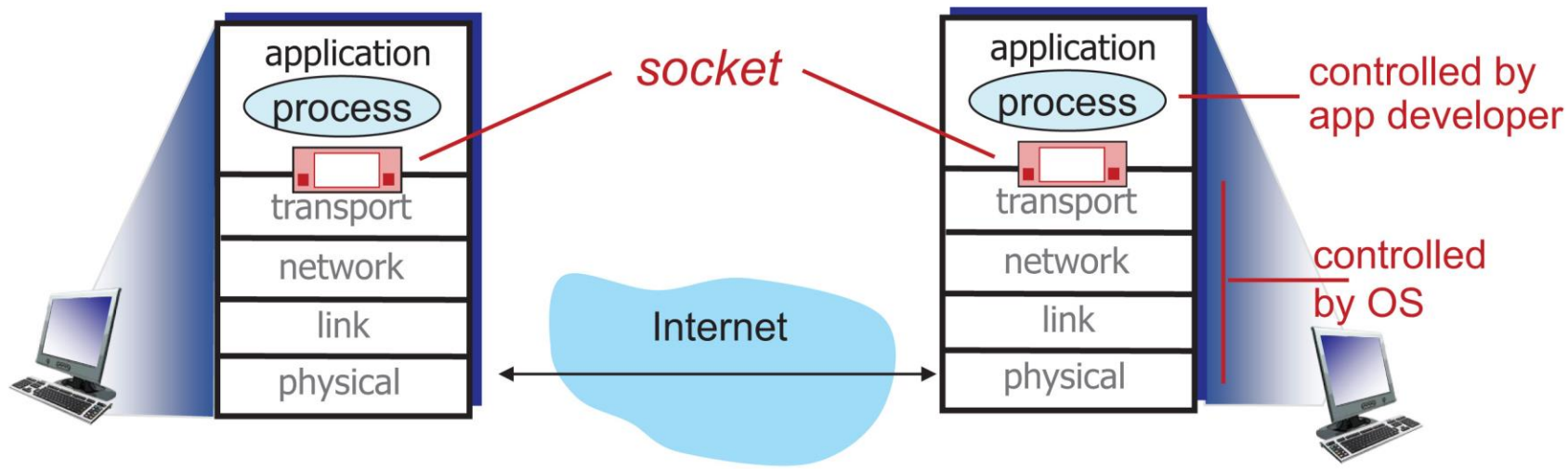
- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- socket programming with UDP and TCP



Socket Programming (1 of 2)

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Socket Programming (2 of 2)

Two socket types for two transport services:

- **UDP:** unreliable datagram
- **TCP:** reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket Programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides **unreliable** transfer of groups of bytes (“datagrams”) between client and server processes

Client/Server Socket Interaction: UDP



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client



create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with serverIP address
And port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Example App: UDP Client

Python UDP Client

include Python's socket library

```
→ from socket import *  
   serverName = 'hostname'  
   serverPort = 12000
```

create UDP socket

```
→ clientSocket = socket(AF_INET, SOCK_DGRAM)
```

get user keyboard input

```
→ message = input('Input lowercase sentence:')
```

attach server name, port to
message; send into socket

```
→ clientSocket.sendto(message.encode(), (serverName, serverPort))
```

read reply data (bytes) from socket

```
→ modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
```

print out received string and close socket

```
→ print(modifiedMessage.decode())  
   clientSocket.close()
```

Note: this code update (2023) to Python 3

Example App: UDP Server

Python UDP Server

create UDP socket

bind socket to local
port number 12000

loop forever

Read from UDP socket into
message, getting client's
address (client IP and port)

send upper case string back to this
client

Note: this code update (2023) to Python 3

```
from socket import *
serverPort = 12000
→ serverSocket = socket(AF_INET, SOCK_DGRAM)
→ serverSocket.bind(('', serverPort)) print('The server is
ready to receive')
→ while True:
→ message, clientAddress = serverSocket.recvfrom(2048)
modifiedMessage = message.decode().upper()
→ serverSocket.sendto(modifiedMessage.encode(),
clientAddress)
```

Socket Programming With TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- **when client creates socket:** client TCP establishes connection to server TCP

- when contacted by client, **server TCP creates new socket** for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)

Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Client/Server Socket Interaction: TCP



server (running on `hostid`)

client



create socket,
port=`x`, for incoming
request:
`serverSocket = socket()`

wait for incoming
connection request
`connectionSocket =`
`serverSocket.accept()`

read request from
`connectionSocket`

write reply to
`connectionSocket`

close
`connectionSocket`

TCP
connection setup

create socket,
connect to `hostid`, port=`x`
`clientSocket = socket()`

send request using
`clientSocket`

read reply from
`clientSocket`

close
`clientSocket`

Example App: TCP Client

Python TCP Client

create TCP socket for server, remote port 12000

No need to attach server name, port

Note: this code update (2023) to Python 3

```
from socket import *
serverName = 'servername'
serverPort = 12000
→ clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())

→ modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Example App: TCP Server

Python TCP Server

create TCP welcoming socket	→	<pre>from socket import * serverPort = 12000 serverSocket = socket(AF_INET,SOCK_STREAM) serverSocket.bind(('',serverPort))</pre>
server begins listening for incoming TCP requests	→	<pre>serverSocket.listen(1) print('The server is ready to receive')</pre>
loop forever	→	<pre>while True:</pre>
server waits on accept() for incoming requests, new socket created on return	→	<pre>connectionSocket, addr = serverSocket.accept()</pre>
read bytes from socket (but not address as in UDP)	→	<pre>sentence = connectionSocket.recv(1024).decode() capitalizedSentence = sentence.upper() connectionSocket.send(capitalizedSentence. encode())</pre>
close connection to this client (but not welcoming socket)	→	<pre>connectionSocket.close()</pre>

Note: this code update (2023) to Python 3

Chapter 2: Summary (1 of 2)

our study of network application layer is now complete!

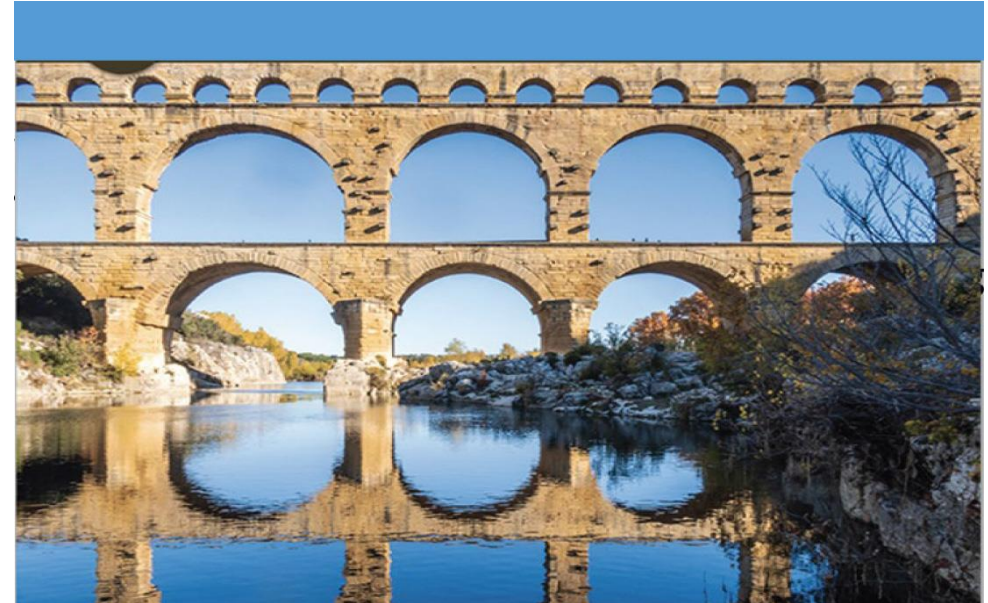
- application architectures
 - client-server
 - P2P
- application service requirements:
 - reliability, bandwidth, delay
- Internet transport service model
 - connection-oriented, reliable: TCP
 - unreliable, datagrams: UDP
- specific protocols:
 - HTTP
 - SMTP, IMAP
 - DNS
- socket programming:
TCP, UDP sockets

Chapter 2: Summary (2 of 2)

Most importantly: learned about **protocols!**

Important themes:

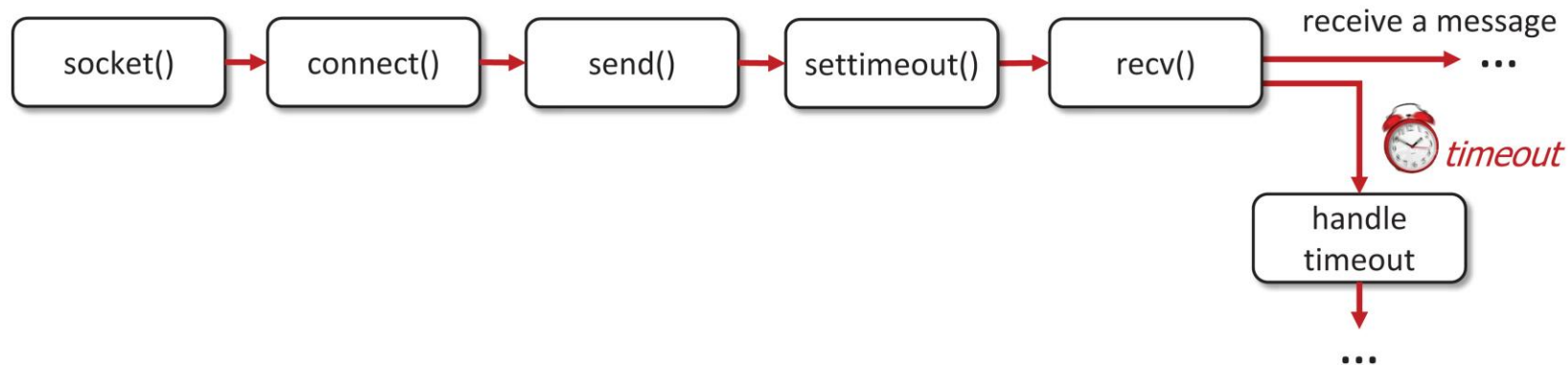
- typical client/server request/reply message exchange
- centralized vs decentralized
- stateless vs stateful
- scalability
- reliable vs unreliable message transfer
- “complexity at network edge”



Additional Chapter 2 Slides

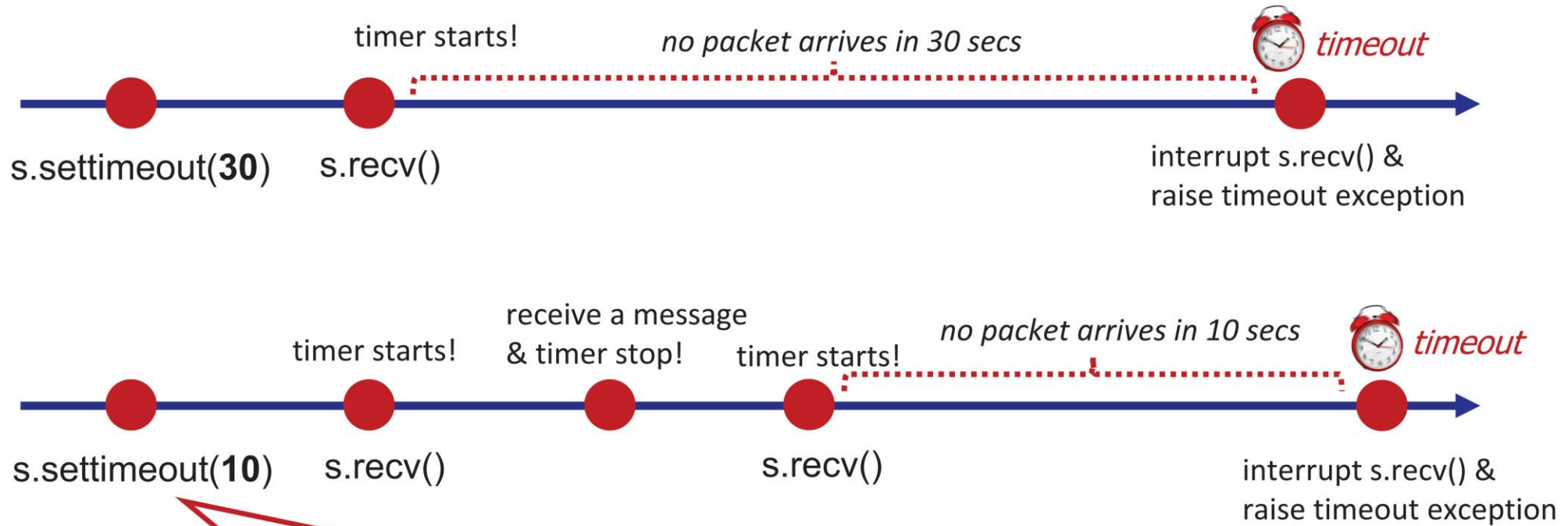
Socket Programming: Waiting for Multiple Events

- sometimes a program must **wait for one of several events** to happen, e.g.,:
 - wait for either (i) a reply from another end of the socket, or (ii) timeout: **timer**
 - wait for replies from several different open sockets: **select()**, **multithreading**
- timeouts are used extensively in networking
- using timeouts with Python socket:



Show example: connect wordle client to random port, it will freeze. handle exception throw exception

How Python `socket.settimeout()` Works?



Set a timeout on all future socket operations of that specific socket!

Python Try-Except Block

Execute a block of code, and handle “exceptions” that may occur when executing that block of code

try:
 <do something>



Executing this **try code block** may cause exception(s) to catch. If an exception is raised, execution jumps from jumps directly into **except code block**

except
<exception>:
 <handle the
 exception>



this **except code block** is only executed if an **<exception>** occurred in the **try code block** (note: except block is **required** with a try block)

Socket Programming: Socket Timeouts

Toy Example:

- A shepherd boy tends his master's sheep.
- If he sees a wolf, he can send a message to villagers for help using a TCP socket.
- The boy found it fun to connect to the server without sending any messages. But the villagers don't think so.
- And they decided that if the boy connects to the server and doesn't send the wolf location **within 10 seconds for three times**, they will **stop listening** to him forever and ever.

set a 10-seconds timeout on all future socket operations

timer starts when `recv()` is called and will raise timeout exception if there is no message within 10 seconds.

How to remove `settimeout()`?

Use `settimeout(None)`. `settimeout(0)` will result in changing the socket to non-blocking mode.



Python TCP Server (Villagers)

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
counter = 0
while counter < 3:
    connectionSocket, addr = serverSocket.accept()
    connectionSocket.settimeout(10)

    try:
        wolf_location =
            connectionSocket.recv(1024).decode()
        send_hunter(wolf_location) # a villager function
        connectionSocket.send('hunter sent')

    except timeout: counter += 1 connectionSocket.close()
```

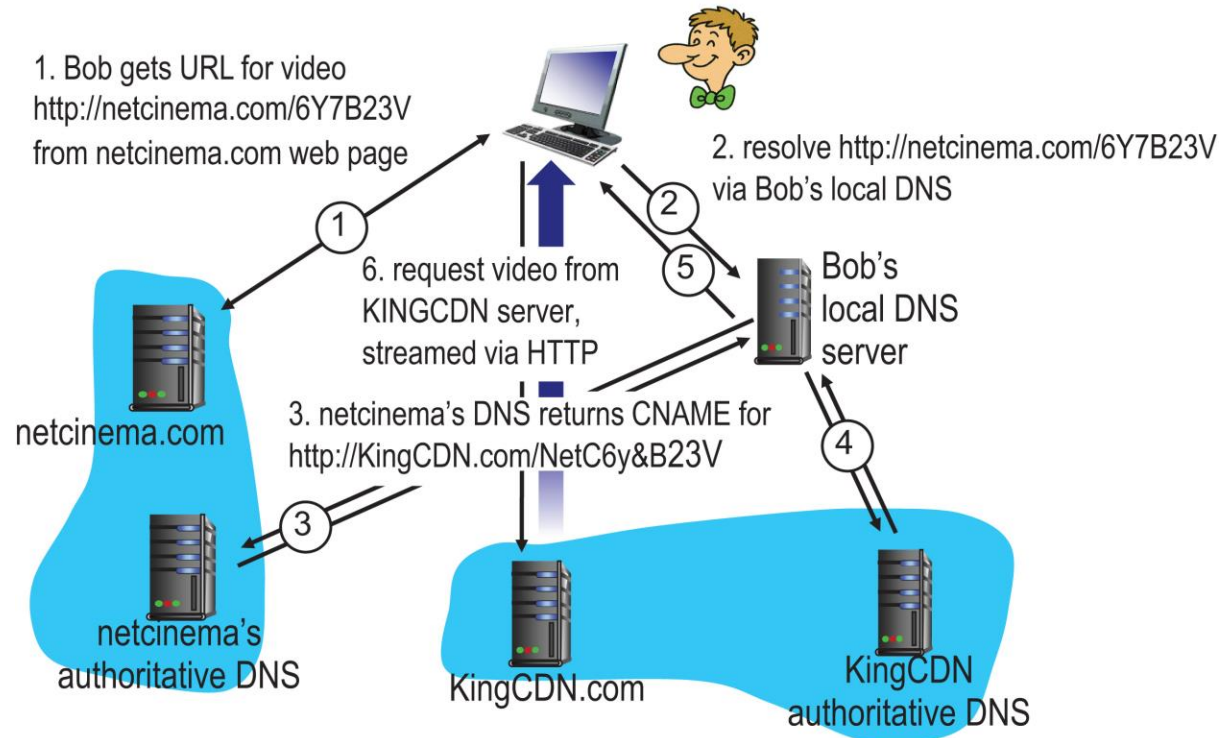
Sample SMTP Interaction (2 of 2)

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

CDN Content Access: A Closer Look

Bob (client) requests video <http://netcinema.com/6Y7B23V>

- video stored in CDN at <http://KingCDN.com/NetC6y&B23V>



Case Study: Netflix

